

Sherwood Wallet — AI Fable 5 Security Review

A security review of the Sherwood Wallet browser extension, focused on fund safety: whether the extension can drain user funds, exfiltrate private keys, or move value without the user's explicit consent.

DATE

September 2026

TYPE

AI Fable 5 security review

Disclaimer

This document is an **AI Fable 5 security review** conducted by Anthropic Fable 5. It is **not a substitute for a full, independent, external audit** and should not be represented as one.

The extension is a self-custodial wallet that deposits into, swaps inside, and withdraws out of the Sherwood shielded vault. The **on-chain Sherwood contracts** and the **Privacy Cash core-evm** primitives they extend are reviewed separately (see the *Sherwood — AI Fable 5 Security Review* of the contracts, and the prior independent external audits by Zigtur, HashCloak, Kriko, and Nethermind of the base pool).

This review deliberately targets the browser extension itself — the code that generates and stores keys, signs transactions, injects a provider into web pages, and talks to the network. Its central question is the one every wallet user should ask: *can this extension steal my money or my keys?* It does not re-derive assurance for the on-chain protocol or the ZK circuits, except where the extension's handling of them affects client-side safety.

Scope

The following extension modules were in scope for this review. The full source is published on GitHub at [sherwood-cash/wallet-extension](https://github.com/sherwood-cash/wallet-extension):

MODULE	PATH	ROLE
Keystore / vault	<code>src/wallet/vault.ts</code> , <code>src/wallet/useVault.ts</code>	Key generation, encryption at rest, unlock, auto-lock, signing
Manifest & CSP	<code>manifest.json</code>	Permissions, host access, content-security-policy, web-accessible resources
Service worker	<code>src/dapp/background.ts</code>	Injected-provider hub: connect/sign/send routing, chain guard
Content bridge	<code>src/dapp/content.ts</code> , <code>src/dapp/inpage.ts</code>	<code>window.ethereum</code> (EIP-1193 / EIP-6963) page bridge
Approval popup	<code>src/dapp/Approval.tsx</code>	User-confirmed signing for dapp requests
Send / Withdraw / Swap	<code>src/screens/Send.tsx</code> , <code>Withdraw.tsx</code> , <code>Swap.tsx</code> , <code>Deposit.tsx</code>	Fund-moving flows
Stealth receive	<code>src/screens/StealthReceive.tsx</code> , <code>src/protocol/lib/stealth/*</code>	One-time receive addresses, shielding to vault
Privacy layer	<code>src/protocol/lib/privacy/*</code>	Notes, proofs, encryption, indexer client
Network clients	<code>src/protocol/lib/rpc.ts</code> , <code>relayer.ts</code> , <code>indexer.ts</code> , <code>stealth/api.ts</code> , <code>src/wallet/rpc.ts</code>	All outbound HTTP/RPC
Build & packaging	<code>vite.config.ts</code> , <code>scripts/*.mjs</code> , <code>package.json</code>	Bundling and supply chain

System Overview

Sherwood Wallet is a Manifest V3 Chrome extension and a **self-custodial** wallet: the user's key is created in the browser, encrypted at rest, and never leaves the device. The wallet operates in two modes. In **Normal mode** it behaves like a conventional EVM wallet on the Robinhood chain (id `4663`) — hold, send, and receive. In **Private mode** it interacts with the Sherwood shielded vault, letting the user deposit, privately swap, and withdraw through relayer-submitted zero-knowledge proofs, plus a **stealth Receive** flow that generates one-time addresses and shields incoming funds into the vault.

The extension also exposes an **injected provider** (`window.ethereum`, EIP-1193 with EIP-6963 discovery) so the same wallet can connect to arbitrary dapps in the MetaMask model. This is the largest trust surface, so its consent model was a focus of the review: a content script bridges the page to a background service worker, and **every** privileged action — connecting an origin, signing a message, sending a transaction — is routed to an approval popup that the user must explicitly confirm. The service worker itself never holds or signs with a raw private key; signing happens only in the popup, backed by the encrypted keystore.

Cryptographic material is handled to the Web3 Secret Storage standard. The seed is generated from a cryptographically secure RNG, encrypted with a password-derived script key, and stored only in extension-local storage. The decrypted key lives in memory-only session storage while unlocked and is cleared on an idle auto-lock. Zero-knowledge circuit artifacts are **bundled with the extension and loaded locally**; nothing is fetched-and-executed at runtime, and a strict content-security-policy forbids remote scripts.

Methodology

The review combined:

- **Manual line-by-line review** of every in-scope module, with particular attention to the four questions that determine whether a wallet is safe to install: where key material lives and whether it can leave the device; where the extension sends network traffic; whether a website can move funds or extract accounts without consent; and whether any code runs that the user did not ship.
 - **Egress analysis.** Every `fetch`, RPC call, and message channel was enumerated and traced to its destination, to confirm no key material is transmitted and no traffic reaches an unexpected host.
 - **Consent-flow analysis** of the injected provider: tracing each dapp method from the page, through the content bridge, into the service worker's routing, to confirm that connect/sign/send cannot bypass the approval popup.
 - **Adversarial verification of each candidate finding by independent agents.** Every observation raised during manual review was re-checked against the actual source by a separate verifier tasked with either reproducing the issue against the code (confirming exact file and line references and reachability) or refuting it. Observations that survived this adversarial pass — including confirmation of their bounded, non-exploitable nature — are the ones recorded below.
-

Severity Classification

SEVERITY	DEFINITION
Critical	Directly exploitable loss of user funds or key material, or silent movement of value, reachable by an untrusted party (a website or the backend).
High	Serious impact (fund loss or key exposure) that requires specific but attainable conditions, or is partially mitigated.
Medium	Meaningful correctness or security defect with a realistic path to harm, but bounded impact or requiring a privileged/unlikely precondition.
Low	Minor correctness, robustness, or hardening issue with no direct path to fund loss or key exposure.
Informational	Observations, defense-in-depth suggestions, documentation notes, or accepted design tradeoffs with no exploitable impact.

Findings Summary

No Critical, High, or Medium severity issues were identified. In particular, the review found no drainer behavior, no key exfiltration, no hidden fund recipients, and no remote or obfuscated code. All confirmed findings are Low or Informational and concern hardening or accepted convenience/security tradeoffs.

ID	TITLE	SEVERITY	STATUS
EXT-01	script work factor lowered from 2^15 to 2^14 for the keystore KDF	Low	Acknowledged
EXT-02	Decrypted key persists in memory-only session storage for up to the idle auto-lock window	Informational	Acknowledged
EXT-03	Content script matches all https://*/* origins to inject the provider bridge	Informational	By design
EXT-04	Optional https://*/* host permission requested at runtime for custom RPC endpoints	Informational	By design
EXT-05	Data endpoints are inlined at build time from environment configuration	Informational	By design

Fund-Safety & Key-Custody Analysis

Because the entire purpose of this review is to answer *"can this extension take my funds or keys,"* the four load-bearing safety properties are stated explicitly, each with the concrete evidence that supports it.

1. Private keys never leave the device

The seed is generated with a cryptographically secure RNG — `ethers.Wallet.createRandom()`, which draws from the platform CSPRNG (`src/wallet/vault.ts`); there is **no use of `Math.random()`** anywhere in the codebase. It is encrypted at rest as a Web3 Secret Storage keystore (scrypt KDF + AES + MAC) via `wallet.encrypt(password, ...)` and written only to `chrome.storage.local`; the decrypted key is written only to `chrome.storage.session` (memory-only, cleared when the browser process ends).

Every outbound request body was inspected across the relayer, indexer, stealth, and RPC clients: they carry only zero-knowledge proofs, external-data hashes, EIP-712/permit signatures, public addresses, and public meta-addresses — **never a private key or mnemonic**. A repository-wide search for logging of secret material returns no matches. The mnemonic reaches only React state, memory-only session storage, and the system clipboard on an **explicit** user copy action. Revealing the secret re-prompts for the password rather than reading it back from session.

2. Network egress is limited to expected hosts

The read provider is a single JSON-RPC endpoint resolved to either the build-configured RPC (defaulting to the Sherwood API's private `/rpc` pass-through), the Robinhood chain RPC, or a **user-configured** custom RPC. A custom RPC is validated to report chain id `4663` before use and requires a runtime host-permission grant. The optional indexer, relayer, and stealth backends resolve their base URLs from environment/deployment configuration and are simply disabled when unset. There is **no hardcoded third-party or attacker host**, and there is no `WebSocket`, `XMLHttpRequest`, `sendBeacon`, `eval`, or `Function()` anywhere in the source.

3. Website interactions are consent-gated (MetaMask model)

The injected provider discloses nothing until the user approves: `eth_accounts` returns an empty array for an unconnected origin, and `eth_requestAccounts` blocks on an approval popup whose promise resolves only after the user clicks. Every signing method — `eth_sendTransaction`, `personal_sign`, `eth_sign`, `eth_signTypedData[_v4]` — first requires the origin to be connected and then routes to an approval popup; a website cannot silently connect, sign, or send. The origin is taken from the browser-supplied sender, not from page-controlled data, so one site cannot impersonate another's connection. Chain switch/add requests are pinned to id `4663`. The service worker never holds a key; it delegates signing to the popup, which uses the keystore signer.

4. No hidden recipients, no remote or obfuscated code

In Send, Withdraw, and Swap, the destination address and amount derive strictly from user input and the user-signed ZK proof. The only additional debit is a **disclosed** relayer/protocol fee, whose recipient is fetched live from the relayer's info endpoint (or a configured protocol-fee recipient) and shown in the UI — not a hardcoded skim address. Stealth shielding provably mints to the user's **own** main-wallet note keys; the one-time stealth

key only signs. There is no `eval`, no `new Function`, no remote dynamic import, and no remote `<script>`; the content-security-policy is `script-src 'self' 'wasm-unsafe-eval'`, which permits only the WASM the prover needs. ZK circuit artifacts are bundled in the extension and loaded from the extension's own origin, never from a remote server. The dependency set is the expected, correctly-spelled canonical set (ethers, snarkjs, `@noble/curves`, `poseidon-lite`, `ffjavascript`, `fixed-merkle-tree`, `qrcode`, React) with no project install hooks.

Detailed Findings

EXT-01 — script work factor lowered from `2^15` to `2^14` for the keystore KDF

Severity: Low **Status:** Acknowledged

Description. The password that protects the on-disk keystore is stretched with scrypt at a work factor of `N = 2^14` (`src/wallet/vault.ts`), rather than the `2^15` used by the default Web3 Secret Storage parameters. The keystore is otherwise standard (scrypt-derived key, authenticated cipher). The value was lowered deliberately to keep unlock responsive inside a popup, where the derivation runs on the main thread.

Impact. No remote or on-chain impact, and no path to fund loss on its own. It marginally reduces the cost of an **offline** brute-force of a **weak** password by an attacker who has already obtained the encrypted keystore file from the victim's disk (i.e. after local machine compromise). A strong password remains infeasible to crack at either work factor. This is a hardening consideration, not an exploitable vulnerability.

Recommendation. Consider raising the work factor back toward `2^15 – 2^17` (optionally running the derivation off the main thread to preserve popup responsiveness), and encourage a strong password at onboarding. The current setting is acceptable given the file is only exposed under local machine compromise.

EXT-02 — Decrypted key persists in memory-only session storage during the unlock window

Severity: Informational **Status:** Acknowledged

Description. While the wallet is unlocked, the decrypted key is held in `chrome.storage.session` (memory-only, never written to disk) and in one live signer object, so the user is not re-prompted for their password on every action. An idle auto-lock (default 30 minutes) clears the session and the cached sign-in signature; the key is also gone when the browser process exits.

Impact. None remotely exploitable. The decrypted key is reachable only by code already running in the extension's own trusted context during the unlock window; it is never exposed to web pages, the content script, or the service worker (which never reads the session key). This is the standard convenience/exposure tradeoff every unlocked hot wallet makes.

Recommendation. Keep the auto-lock window conservative and make it user-configurable; optionally offer a "lock immediately" control. No change is required for safety.

EXT-03 — Content script matches all `https://*/*` origins to inject the provider bridge

Severity: Informational **Status:** By design

Description. To act as an injected wallet on any dapp (the MetaMask model), the content script is registered on `https://*/*` and injects the in-page provider at `document_start` (`manifest.json`, `src/dapp/content.ts`). Broad content-script matches increase the review surface of any extension.

Impact. Bounded. The content script only injects the bundled in-page provider and relays `postMessage` envelopes to the background; it performs no privileged action itself, executes no page-supplied code, and cannot bypass the approval gates, which are re-checked in the service worker on every call. It validates that messages originate from the page's own window.

Recommendation. Retain as required for provider functionality; keep the bridge strictly limited to relaying provider RPC (as it is today). Document the broad match in the store listing's permission justification.

EXT-04 — Optional `https://*/*` host permission requested at runtime for custom RPC

Severity: Informational **Status:** By design

Description. The manifest keeps its baseline `host_permissions` narrow (the Robinhood chain RPC and the Sherwood API) and declares `https://*/*` only as an **optional** host permission, requested at runtime **if and when** the user configures a custom RPC endpoint on an arbitrary origin (`src/wallet/rpc.ts`).

Impact. None by default: the broad host permission is not granted at install and is only ever requested in direct response to a user action (setting their own RPC). Reviewers should not treat the optional entry as standing access.

Recommendation. No change required. This is the correct, least-privilege pattern for a user-configurable RPC.

EXT-05 — Data endpoints are inlined at build time from environment configuration

Severity: Informational **Status:** By design

Description. The default RPC, relayer, indexer, and stealth base URLs are read from build-time environment variables (with deployment-JSON fallbacks) and inlined into the bundle (`vite.config.ts` , `.env`). These are **data** endpoints, not secrets, and are documented as such.

Impact. None. The inlined values point at the Sherwood API and chain RPC; no secret key or token is embedded. A user can override the RPC at runtime.

Recommendation. Continue to keep only non-secret data endpoints in the build environment, and ensure any future secrets remain server-side. No change required.

Conclusion & Recommendations

This review of the Sherwood Wallet browser extension found **no Critical, High, or Medium severity issues**, and — directly answering the question that matters most for a wallet — **no drainer behavior, no key exfiltration, no hidden fund recipients, and no remote or obfuscated code**. Every confirmed finding is **Low** or **Informational** and concerns hardening or an accepted, documented tradeoff.

The extension's safety architecture holds up under scrutiny across the four properties that determine whether a wallet can be trusted:

1. **Key custody** — the seed is generated from a secure RNG, encrypted at rest to the Web3 Secret Storage standard, and the decrypted key lives only in memory-only session storage; it is never transmitted, logged, placed in a URL, written to disk in plaintext, or exposed to web pages or the service worker.
2. **Egress** — outbound traffic reaches only the Sherwood API, the chain RPC, or a user-configured (chain-id-verified) custom RPC; nothing carries key material and no third-party host is baked in.
3. **Consent** — the injected provider follows the MetaMask model: no account disclosure without a connect approval, and no signing or sending without an explicit approval-popup click, with the service worker never handling a raw key.
4. **Integrity** — fund-moving flows send only to user-supplied or disclosed relayer/protocol-fee addresses, ZK circuits are bundled and loaded locally, a strict CSP forbids remote scripts, and the dependency set is clean.

We recommend, in rough priority order: **EXT-01** — consider raising the script work factor (off-thread) and encouraging strong passwords; **EXT-02** — keep the auto-lock conservative and user-configurable; and **EXT-03/04/05** — retain the current least-privilege permission model and document the broad content-script match and optional host permission in the store listing. None of these affects the core conclusion that the extension is safe to install and does not put user funds or keys at risk.

Reminder: this AI Fable 5 security review does not replace a full independent external audit, and a clean result here is not a guarantee of the absence of vulnerabilities.